

Convolution neural networks

Unit-5

CNN

- The human sense of vision is unbelievably advanced. Within fractions of seconds, we can identify objects within our field of view, without thought or hesitation.
- our brain transforms that information into more meaningful primitives—lines, curves, and shapes.
- Foundational to the human sense of vision is the neuron.
- Specialized neurons are responsible for capturing light information in the human eye.
- This light information is then preprocessed, transported to the visual cortex of the brain, and then finally analyzed to completion.
- Neurons are single-handedly responsible for all of these functions

Feature Selection

- How do we teach a computer that an image contains a face? We could try to train a traditional machine learning algorithm by giving it the raw pixel values of the image and hoping it can find an appropriate classifier.
- a human would choose the features (perhaps hundreds or thousands) that he or she believed were important in making a classification decision

- The machine learning algorithm would then use these new feature vectors to make classification decisions. Because the feature extraction process improves the **signal-to-noise ratio**
- FACE: there is a difference in light intensity between the eye region and the upper cheeks. There is also a difference in light intensity between the nose bridge and the two eyes on either side.
- Machine learning algorithm known as **boosting** is used for this identification.
- **ALEXNET** completely revolutionized the field.

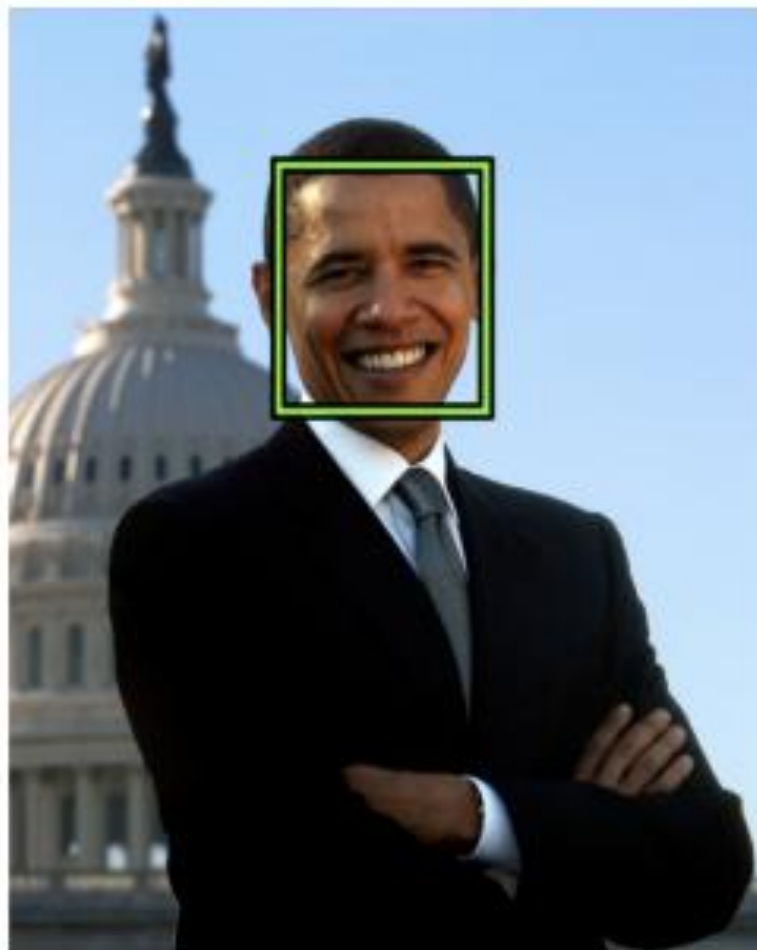


Figure 5-1. A hypothetical face-recognition algorithm should detect a face in this photograph of former President Barack Obama

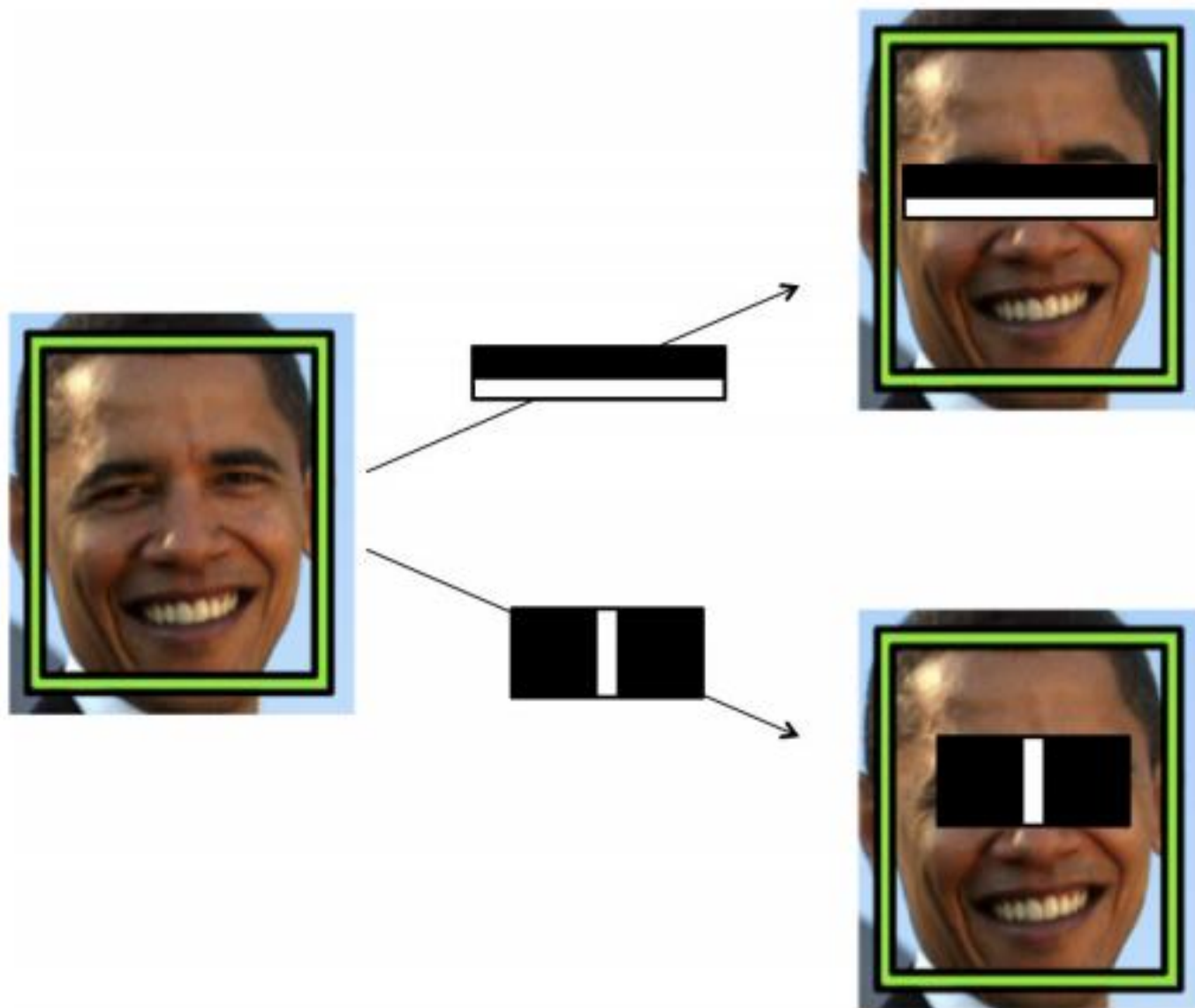


Figure 5-2. An illustration of Viola-Jones intensity detectors

Deep Neural Net

- The fundamental goal in applying deep learning to computer vision is to remove the cumbersome, and ultimately limiting, feature selection process.
- MNIST dataset : 28 x 28 pixels and were black and white.
- neuron in a fully connected hidden layer would have 784 incoming weights

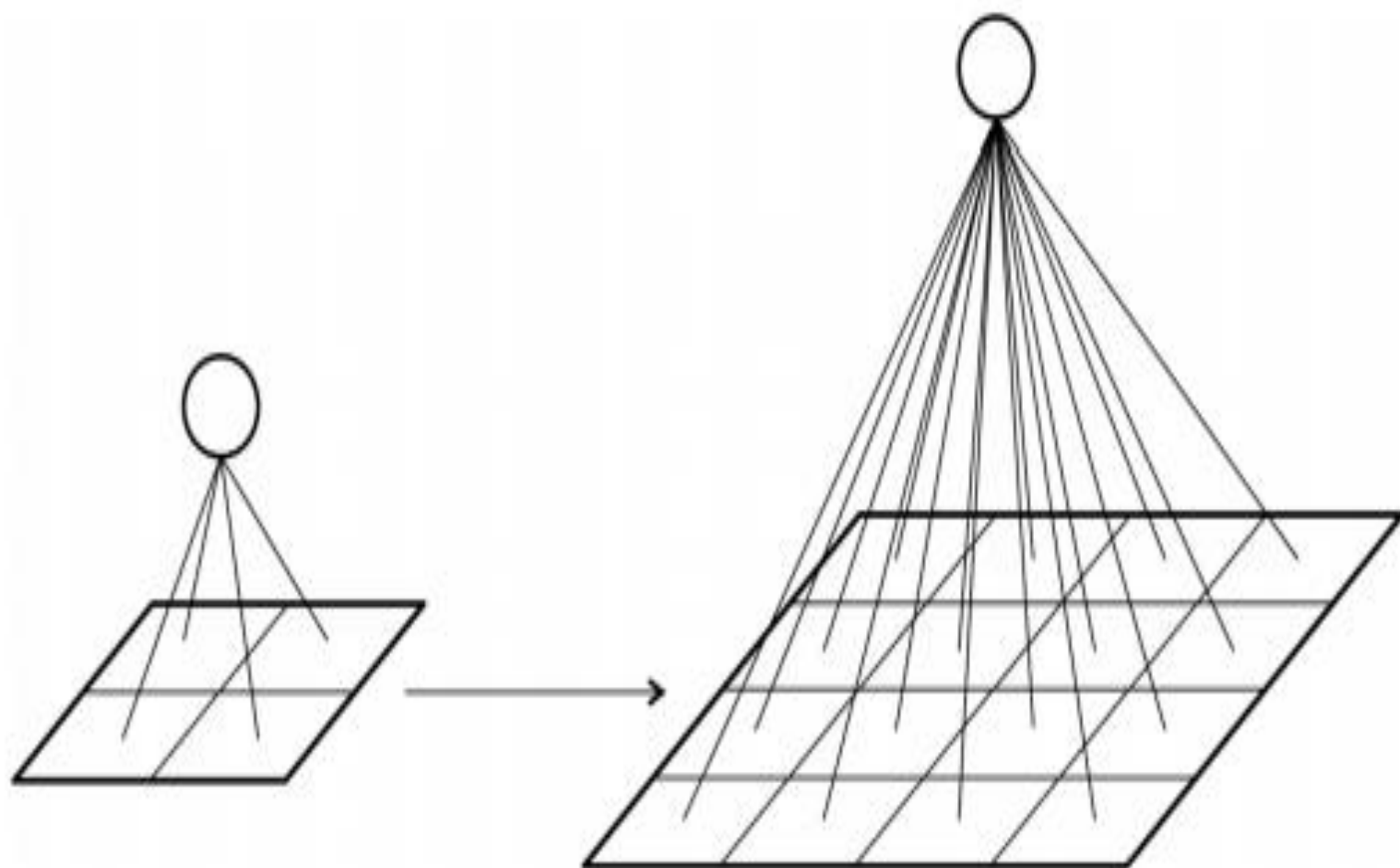


Figure 5-3. The density of connections between layers increases intractably as the size of the image increases

- Inspired by how human vision works, layers of a convolutional network have neurons arranged in three dimensions, so layers have a width, height, and depth.
- neurons in a convolutional layer are only connected to a small, local region of the preceding layer, so we avoid the wastefulness of fully-connected neurons.
- A convolutional layer's function can be expressed simply: it processes a three-dimensional volume of information to produce a new 3D volume of information.

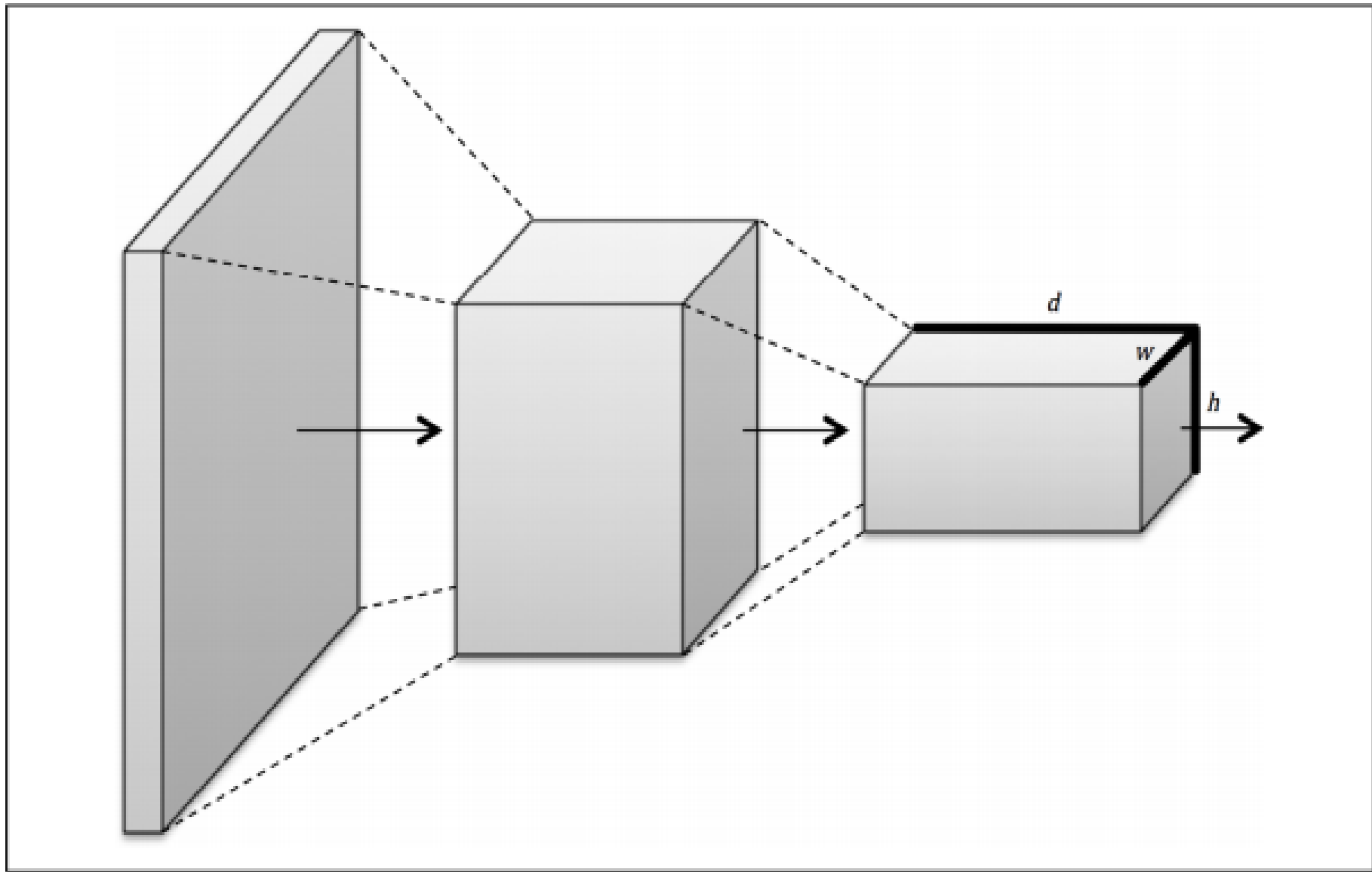
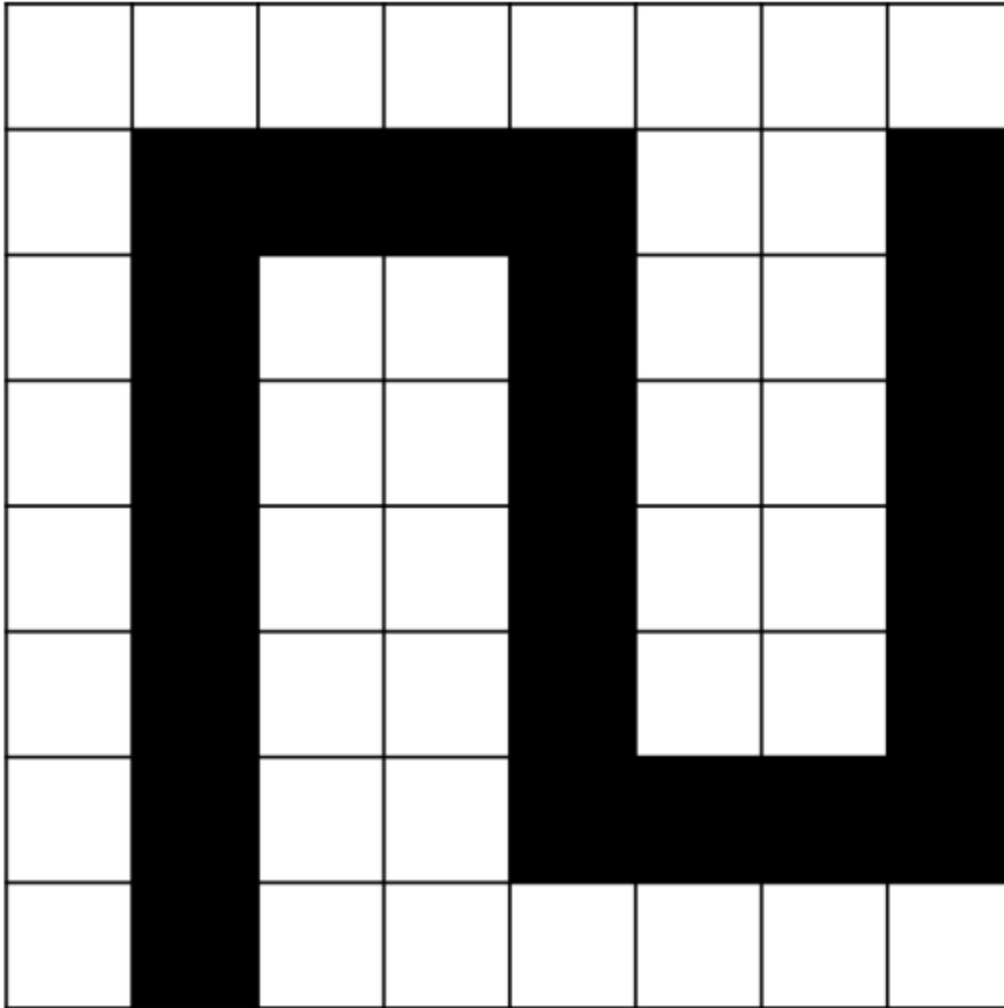


Figure 5-4. Convolutional layers arrange neurons in three dimensions, so layers have width, height, and depth

Filters and Feature Maps

- Parts of the visual cortex are responsible for detecting edges with the help of an experiment.
- **Scientists:-** inserted electrodes into the brain of a cat and projected black-and-white patterns on the neurons fired only when there were vertical lines, others when there were horizontal lines, and still others when the lines were at particular angles.
- visual cortex was organized in layers.
- Each layer is responsible for building on the features detected in the previous layers—from lines, to contours, to shapes, to entire objects.
- Within a layer of the visual cortex, the same feature detectors were replicated over the whole area in order to detect features in all parts of an image.
- These ideas significantly impacted the design of convolutional neural nets.screen.

- A filter is essentially a feature detector, and to understand how it works,



Consider a toy image and how this image

Feature in toy image

- Let's say that we want to detect vertical and horizontal lines in the image.
- One approach would be to use an appropriate **feature detector**.
- to **detect vertical lines, we would use the feature detector** on the top, slide it across the entirety of the image, and at every step check if we have a match.
- We keep track of our answers in the matrix in the top right. If there's a **match**, we **shade the appropriate box black**. If there isn't, we leave it white.
- This result is our feature map
- We can do the same for the horizontal line detector (bottom), resulting in the feature map in the bottom-right corner.

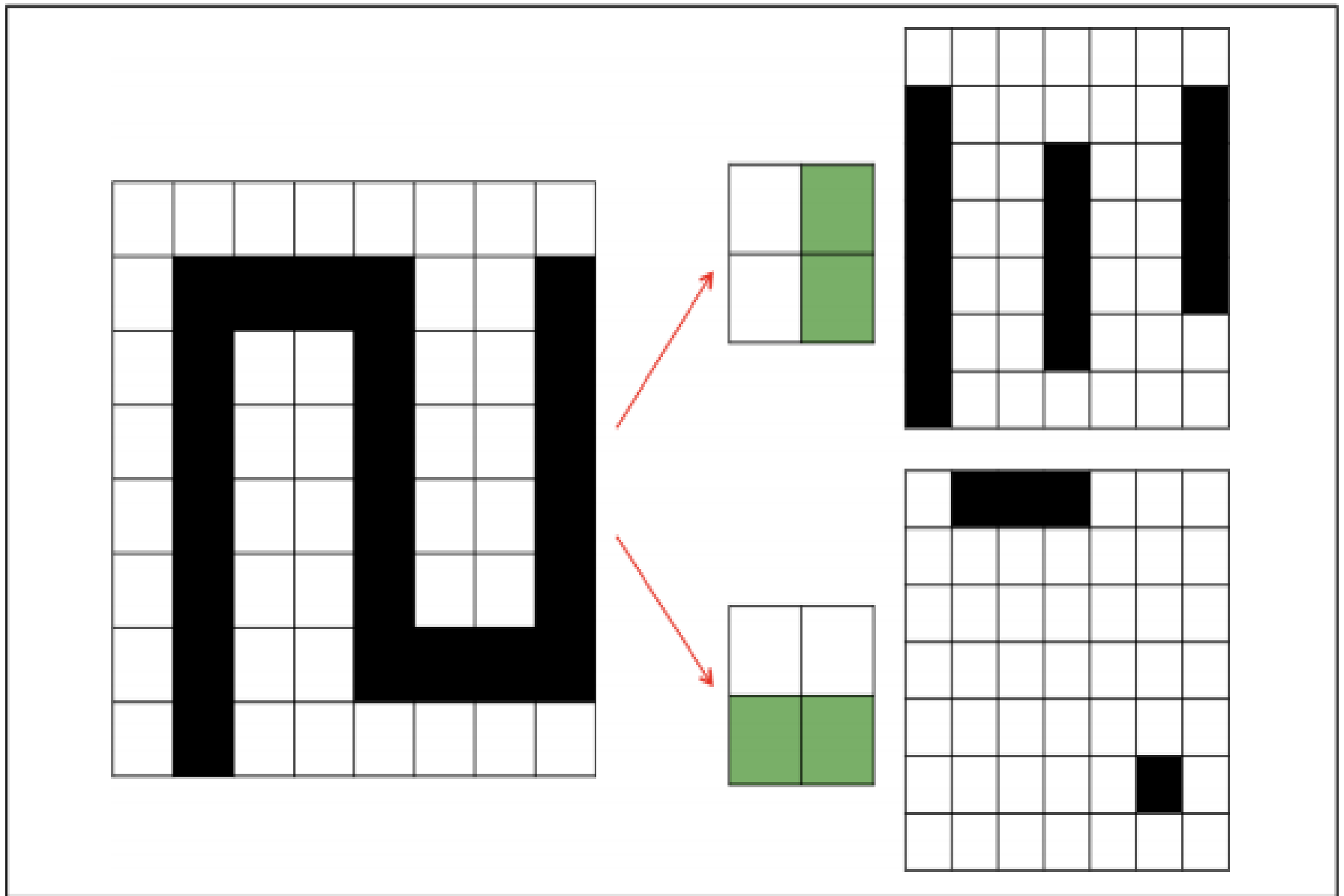


Figure 5-6. Applying filters that detect vertical and horizontal lines on our toy example

- This operation is called a **convolution**.
- We take a filter and we multiply it over the entire area of an input image.
- layers of neurons in a feed forward neural net represent either the **original image or a feature map**.
- Filters represent combinations of connections.
- Connections of the same color are restricted to always have the same weight.
- We can achieve this by initializing all the connections in a group with identical weights and by always averaging the weight updates of a group before applying them at the end of each iteration of back-propagation.
- The output layer is the **feature map** generated by this filter. A neuron in the feature map is activated if the filter contributing to its activity detected an appropriate feature at the corresponding position in the previous layer.

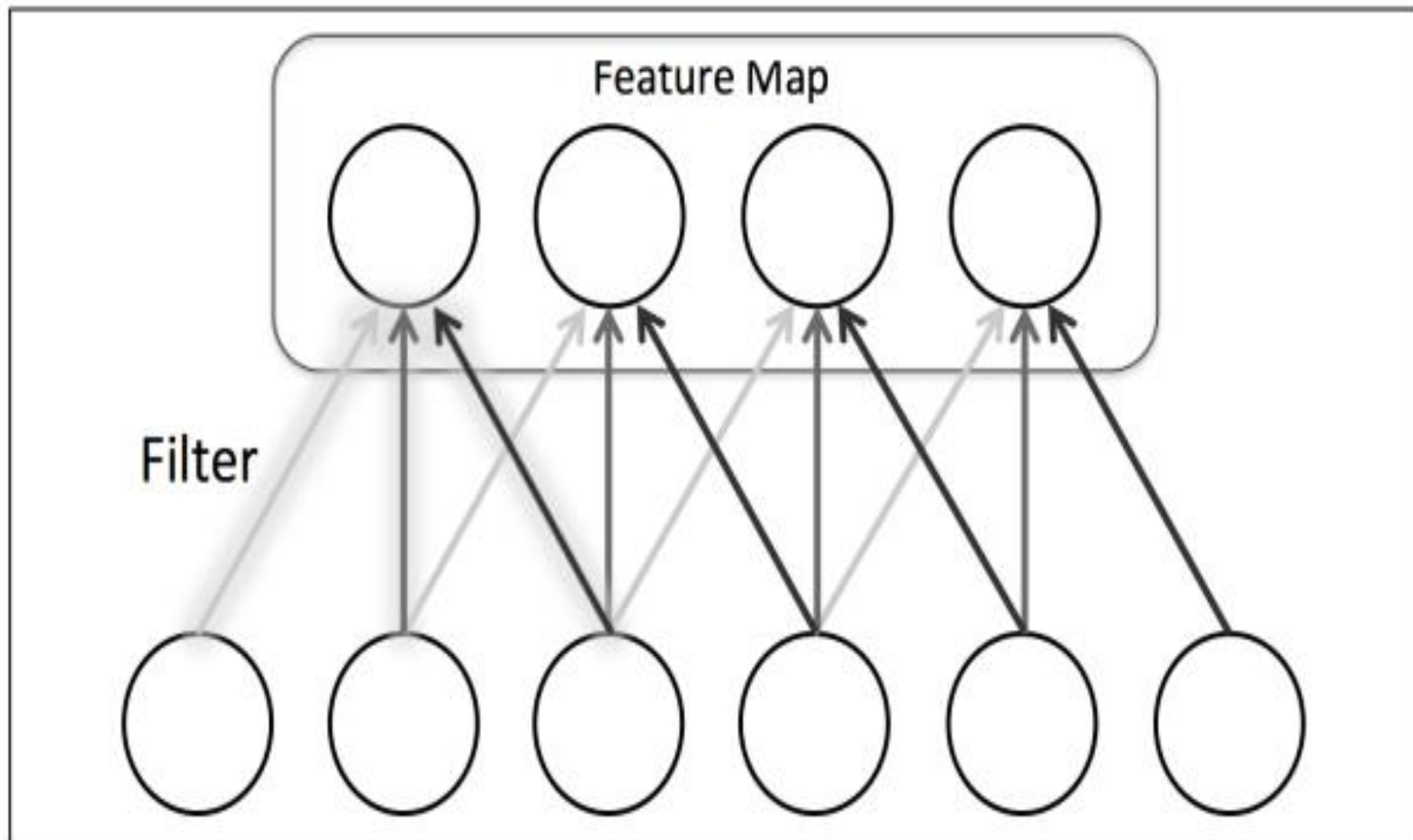


Figure 5-7. Representing filters and feature maps as neurons in a convolutional layer

$$m_{ij}^k = f((W * x)_{ij} + b^k)$$

- They operate on the entire volume of feature maps that have been generated at a particular layer.
- In other words, to make decisions about the existence of a face, we must **combine evidence over multiple feature maps**.
- This is equally necessary for an input image that is of full color. These images have pixels represented as RGB values, we require three slices in the input volume (one slice for each color).
- feature maps must be able to operate over volumes, not just areas.

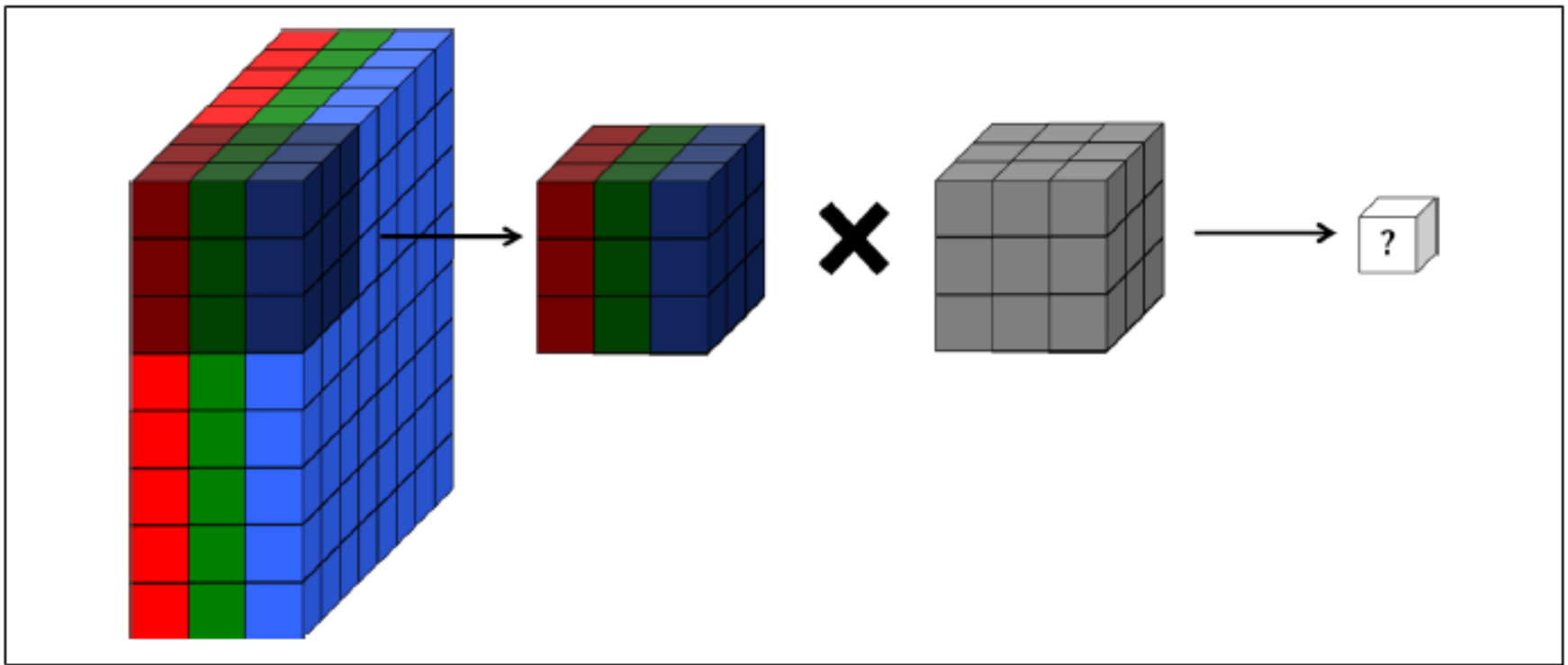


Figure 5-8. Representing a full-color RGB image as a volume and applying a volumetric convolutional filter

- Each cell in the input volume is a neuron.
- A local portion is multiplied with a filter (corresponding to weights in the convolution layer) to produce a neuron in a filter map in the following volumetric layer of neurons.

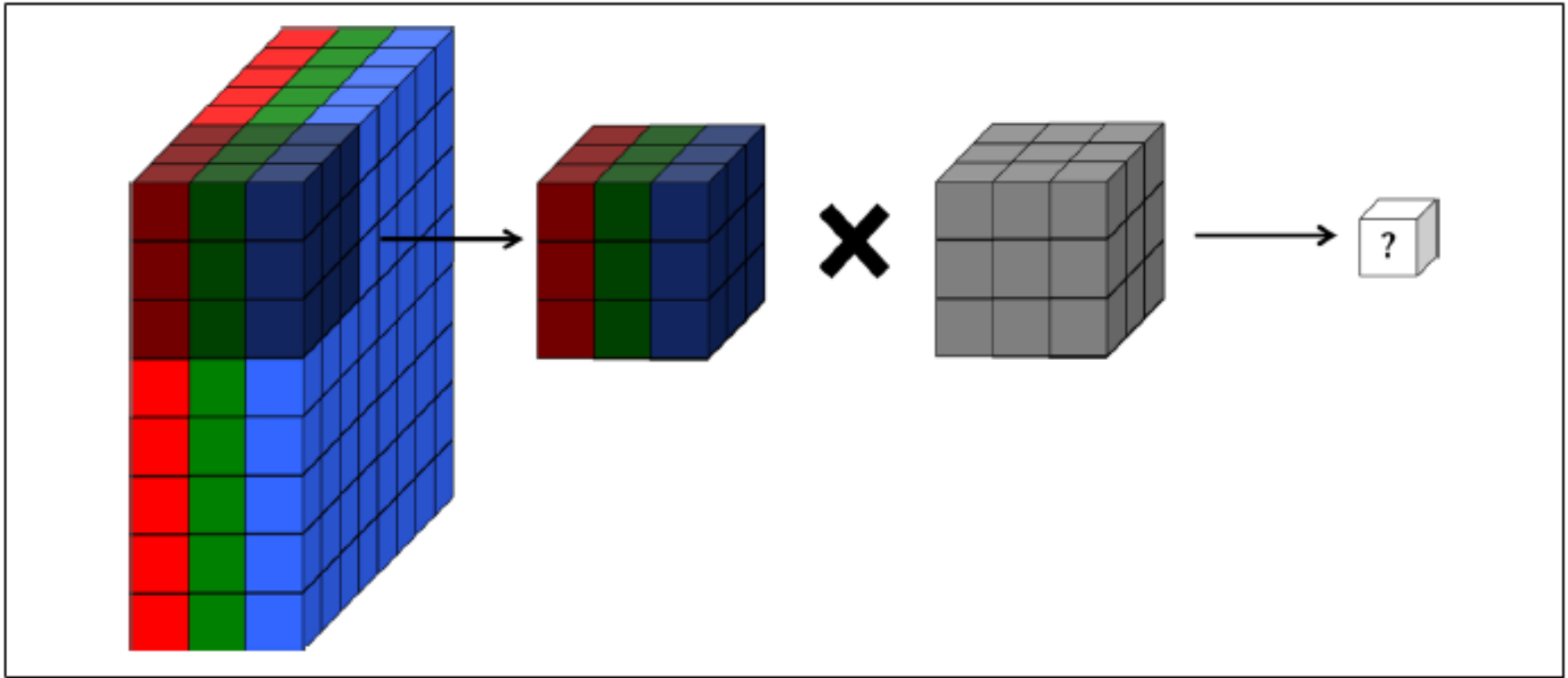


Figure 5-8. Representing a full-color RGB image as a volume and applying a volumetric convolutional filter

- Convolutional layer (which consists of a set of filters) converts one volume of values into another volume of values.
- The depth of the filter corresponds to the depth of the input volume. This is so that the filter can combine information from all the features that have been learned. The depth of the output volume of a convolution layer is equivalent to the number of filters in that layer, because each filter produces its own slice.

The relationships can be depicted in the figure below as :

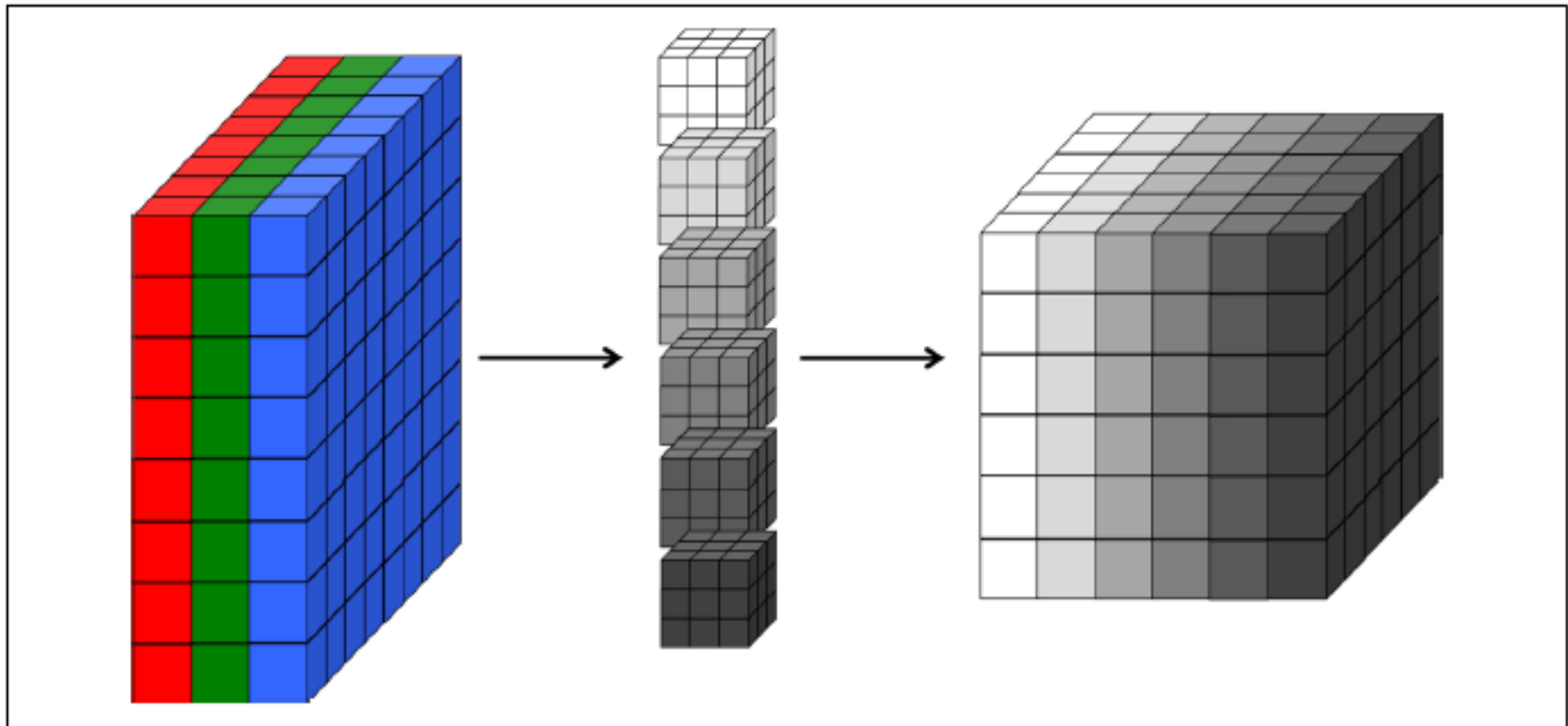


Figure 5-9. A three-dimensional visualization of a convolutional layer, where each filter corresponds to a slice in the resulting output volume

Full Description of the Convolutional Layer

- First, a convolutional layer takes in an input volume.
- This input volume has the following characteristics:
 - Its *width* w_{in}
 - Its *height* h_{in}
 - Its *depth* d_{in}
 - Its *zero padding* p
- This volume is processed by a total of k *filters*, which represent the *weights and connections* in the convolutional network.
- These filters have a number of hyper-parameters, which are described as follows:

- Their *spatial extent 'e', which is equal to the filter's height and width.*
- Their *stride 's', or the distance between consecutive applications of the filter on the input volume.*
- If we use a stride of 1, we get the full convolution.
- The bias b (*a parameter learned like the values in the filter*) which is added to each component of the convolution.

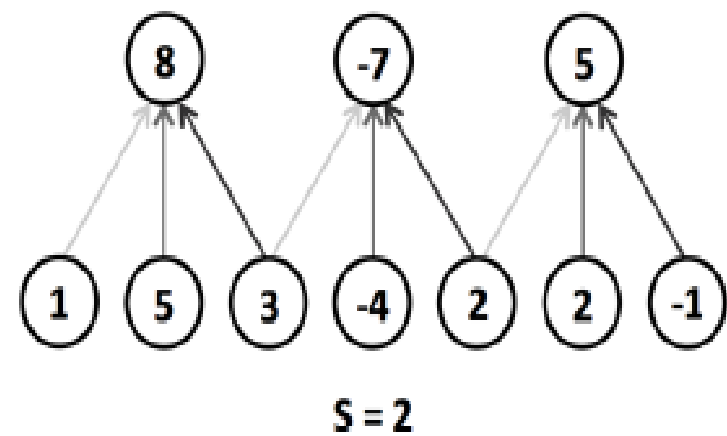
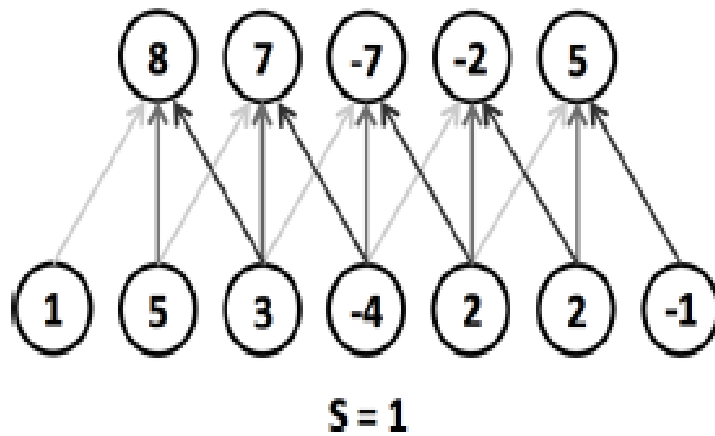
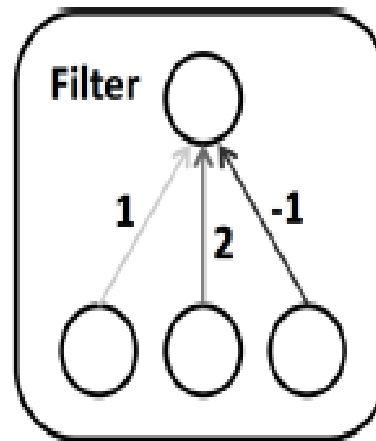


Figure 5-10. An illustration of a filter's stride hyperparameter

This results in an output volume with the following characteristics:

- Its function f , which is applied to the incoming logit of each neuron in the output volume to determine its final value
- Its *width* $w_{out} = \left\lceil \frac{w_{in} - e + 2p}{s} \right\rceil + 1$

$$\text{Its depth } d_{out} = k$$

The m^{th} “depth slice” of the output volume, where $1 \leq m \leq k$, corresponds to the function f applied to the sum of the m^{th} filter convoluted over the input volume and the bias b^m . Moreover, this means that per filter, we have $d_{in}e^2$ parameters. In total, that means the layer has $kd_{in}e^2$ parameters and k biases. To demonstrate this in

with a $5 \times 5 \times 3$ input volume with zero padding $p = 1$. We'll use two $3 \times 3 \times 3$ filters (spatial extent) with a stride $s = 2$. We'll use a linear function to produce the output volume, which will be of size $3 \times 3 \times 2$.

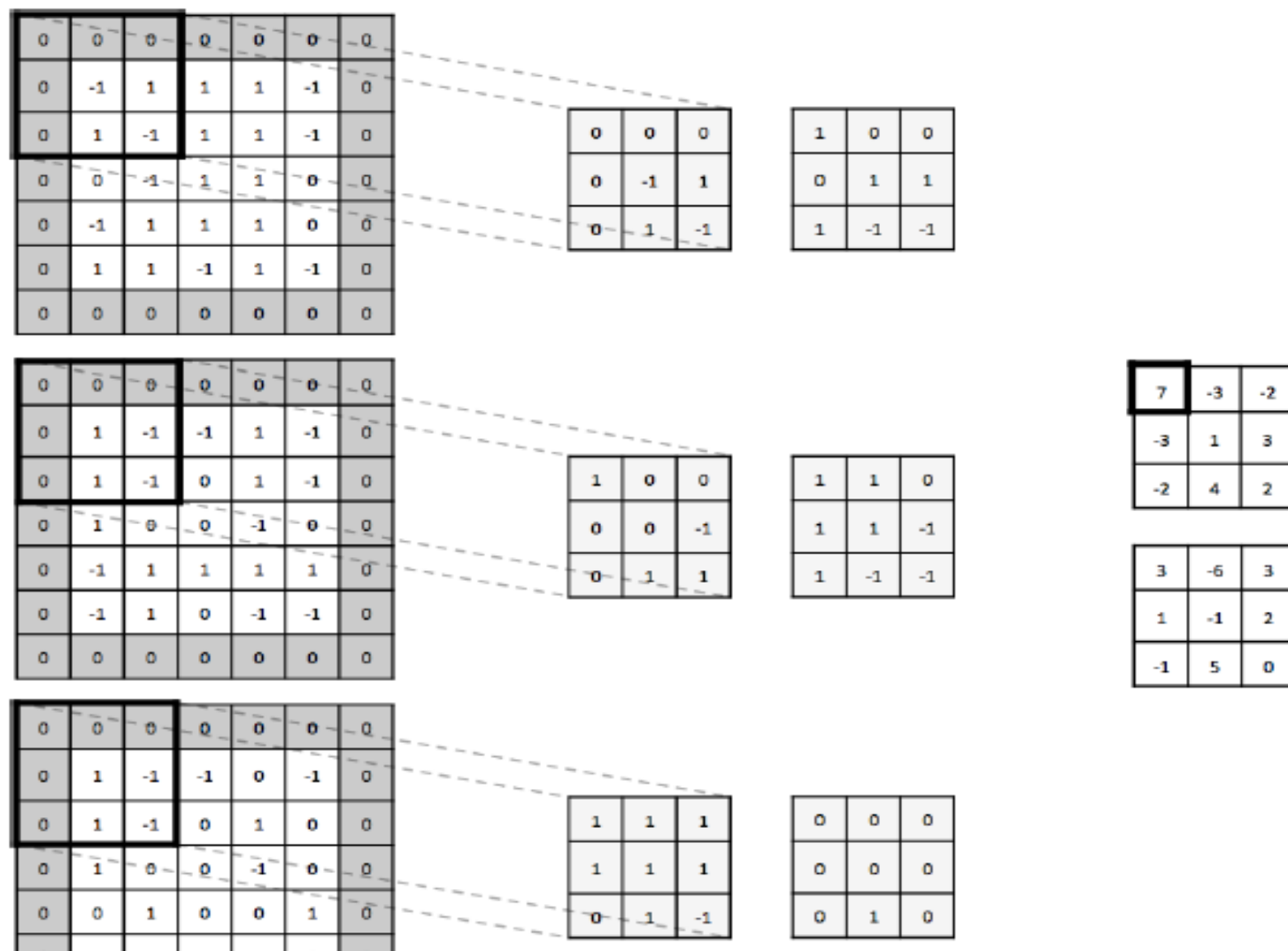


Figure 5-11. This is a convolutional layer with an input volume that has width 5, height 5, depth 3, and zero padding 1. There are 2 filters, with spatial extent 3 and applied with a stride of 2. It results in an output volume with width 3, height 3, and depth 2. We apply the first convolutional filter to the upper-leftmost 3×3 piece of the input volume

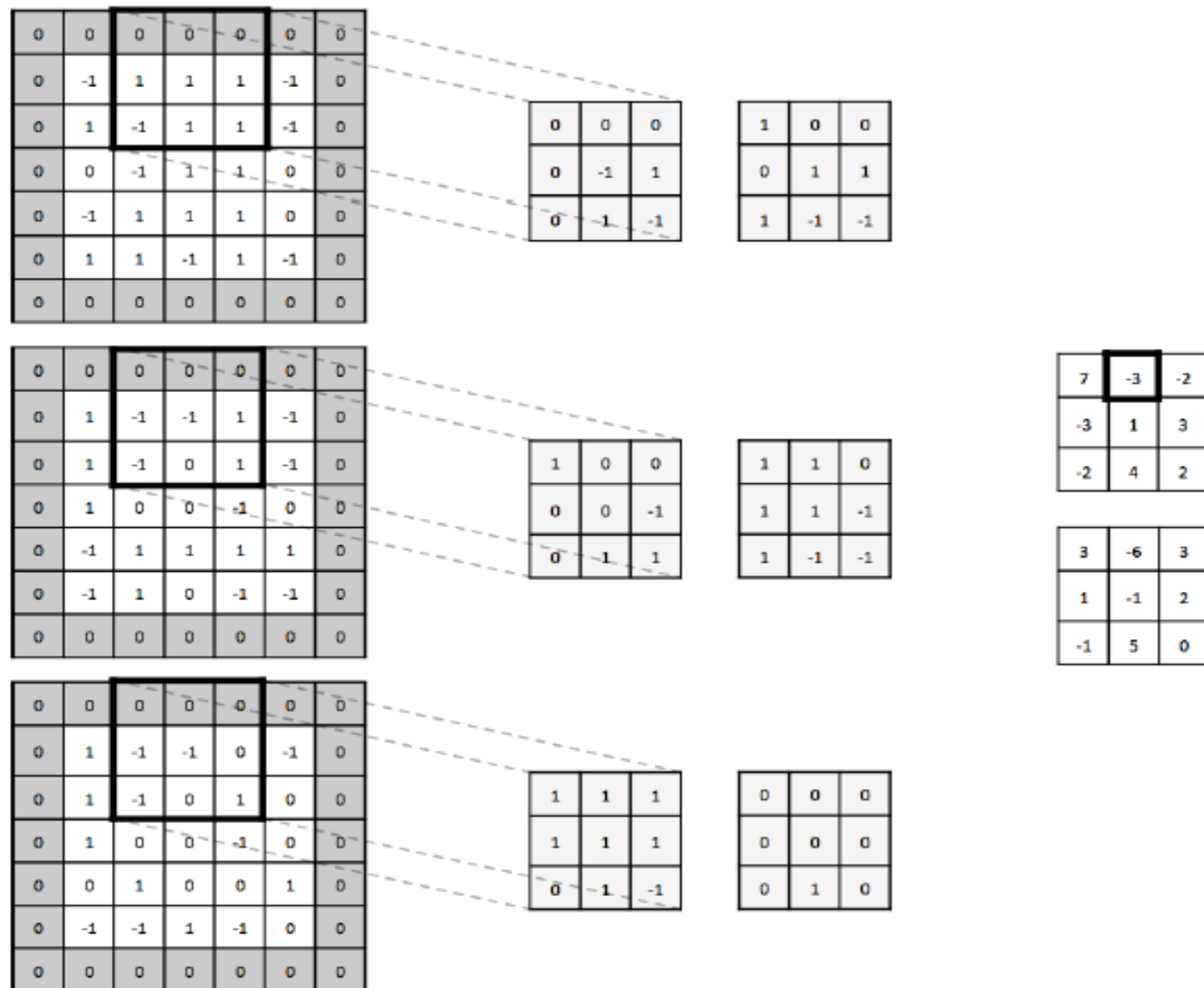


Figure 5-12. Using the same setup as Figure 5-11, we generate the next value in the first depth slice of the output volume

- Generally, it's wise to keep filter sizes small (size 3 x 3 or 5 x 5).
- larger sizes are used (7 x 7) but only in the first convolutional layer.
- Having more small filters is an easy way to achieve **high representational power** while also incurring a smaller number of parameters.
- use a stride of 1 to capture all useful information in the feature maps, and a 0 padding that keeps the output volume's height and width equivalent to the input volume's height and width.
- Tensor Flow provides us with a convenient operation to easily perform a convolution

**`tf.nn.conv2d(input, filter, strides, padding,
use_cudnn_on_gpu=True,name=None)`**

Max Pooling

- To aggressively reduce **dimensionality of feature** maps and sharpen the located features
- *The* essential idea behind max pooling is to break up **each feature map into equally sized tiles**.
- Then we create a condensed feature map.
- we create a cell for each tile, compute the maximum value in the tile, and propagate this maximum value into the corresponding cell of the condensed feature map.

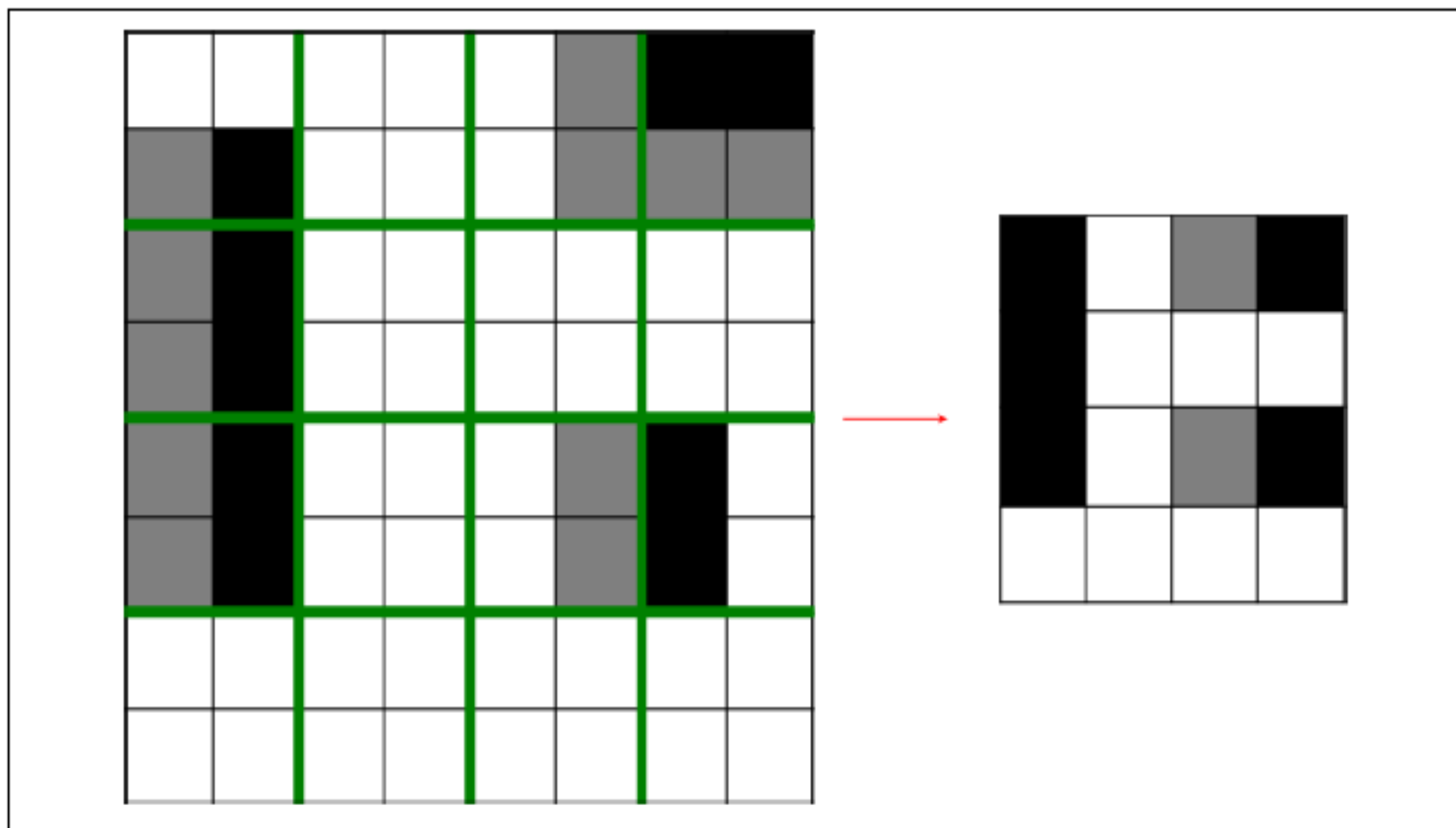
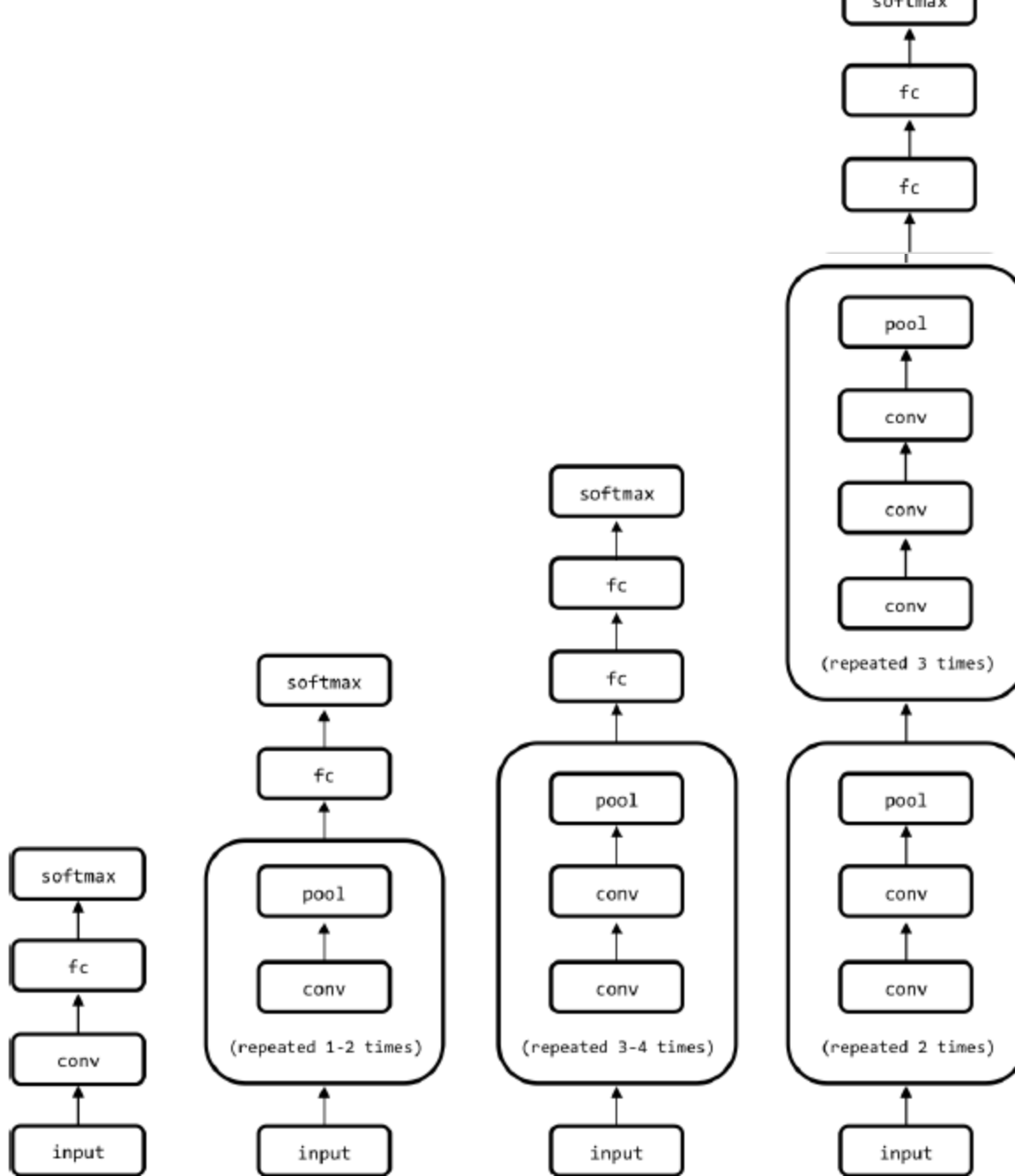


Figure 5-13. An illustration of how max pooling significantly reduces parameters as we move up the network

- More rigorously, we can describe a pooling layer with two parameters:
 1. Its spatial extent e
 2. Its stride s
- The first is the non-overlapping pooling layer with $e = 2, s = 2$.
- *The second is the overlapping pooling layer with $e = 3, s = 2$. The resulting dimensions of each feature map are as follows:*

- Its width $w_{out} = \left\lceil \frac{w_{in} - e}{s} \right\rceil + 1$
- Its height $h_{out} = \left\lceil \frac{h_{in} - e}{s} \right\rceil + 1$

- *locally invariant* : This means that even if the inputs shift around a little bit, the output of the max pooling layer stays constant.
- enforcing large amounts of local invariance can destroy our network's ability to carry important information.
- we usually keep the spatial extent of our pooling layers quite small.
- *fractional max pooling*, a pseudorandom number generator is used to generate tilings with non-integer lengths for pooling.
- Fractional max pooling functions as a strong regularizer, helping prevent over fitting in convolutional networks.



Closing the Loop on MNIST with Convolutional Networks

- In MNIST , our feed-forward network was able to achieve a 98.2% accuracy.
- build a convolutional network with a pretty standard architecture : two pooling and two convolutional interleaved, followed by a fully connected layer (with dropout, $p = 0.5$) *and a terminal softmax. To make building the network easy.*

[illegible]

Procedure

- The first helper method generates a convolutional layer with a particular shape.
- We set the stride to be 1 and the padding to keep the width and height constant between input and output tensors.
- We also initialize the weights using the same heuristic we used in the feed-forward network.
- The number of incoming weights into a neuron spans the filter's height and width and the input tensor's depth.

The second helper method generates a max pooling layer with non-overlapping windows of size k . The default, as recommended, is $k=2$, and we'll use this default in our MNIST convolutional network.

With these helper methods, we can now build a new inference constructor:

```
def inference(x, keep_prob):

    x = tf.reshape(x, shape=[-1, 28, 28, 1])
    with tf.variable_scope("conv_1"):
        conv_1 = conv2d(x, [5, 5, 1, 32], [32])
        pool_1 = max_pool(conv_1)

    with tf.variable_scope("conv_2"):
        conv_2 = conv2d(pool_1, [5, 5, 32, 64], [64])
        pool_2 = max_pool(conv_2)

    with tf.variable_scope("fc"):
        pool_2_flat = tf.reshape(pool_2, [-1, 7 * 7 * 64])
        fc_1 = layer(pool_2_flat, [7*7*64, 1024], [1024])

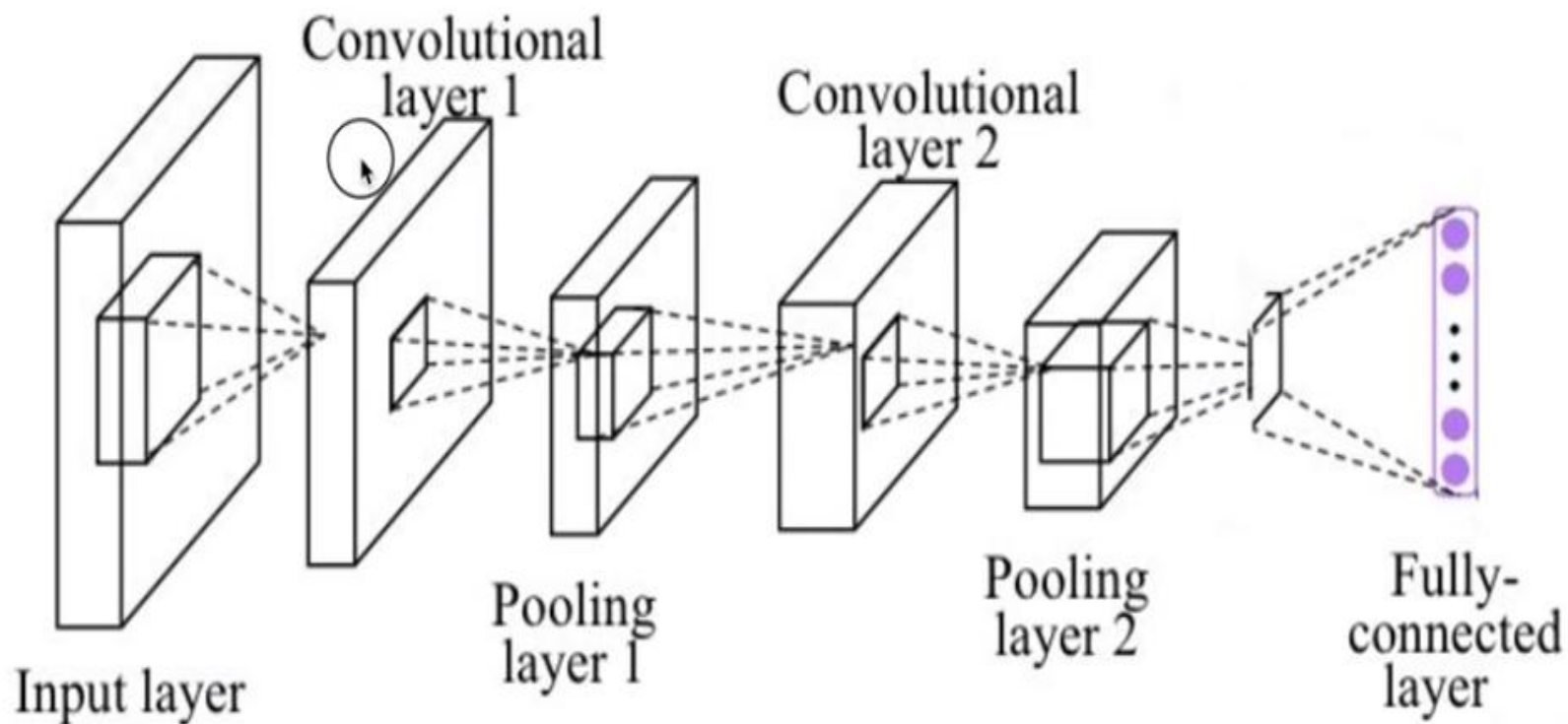
        # apply dropout
        fc_1_drop = tf.nn.dropout(fc_1, keep_prob)

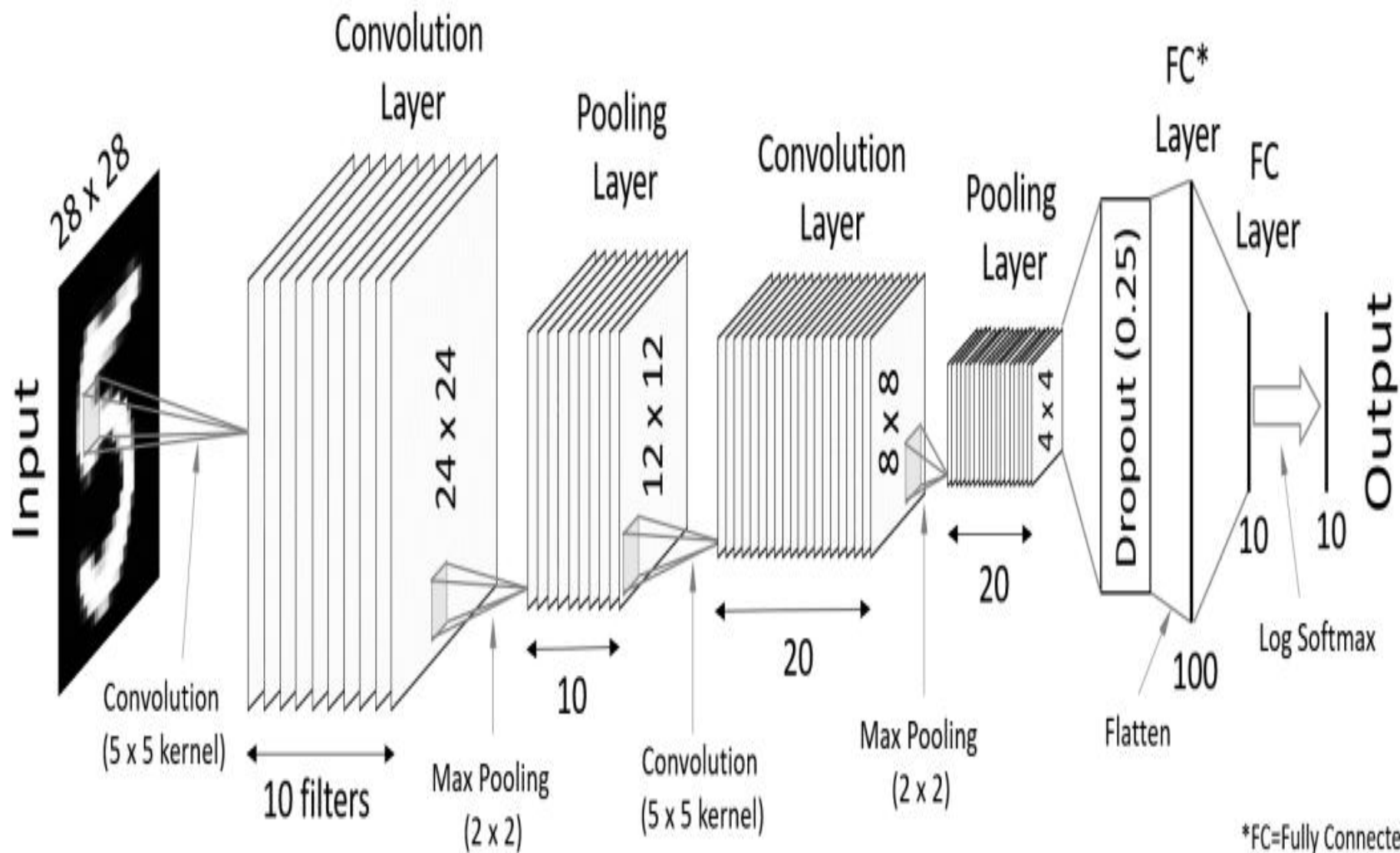
    with tf.variable_scope("output"):
        output = layer(fc_1_drop, [1024, 10], [10])

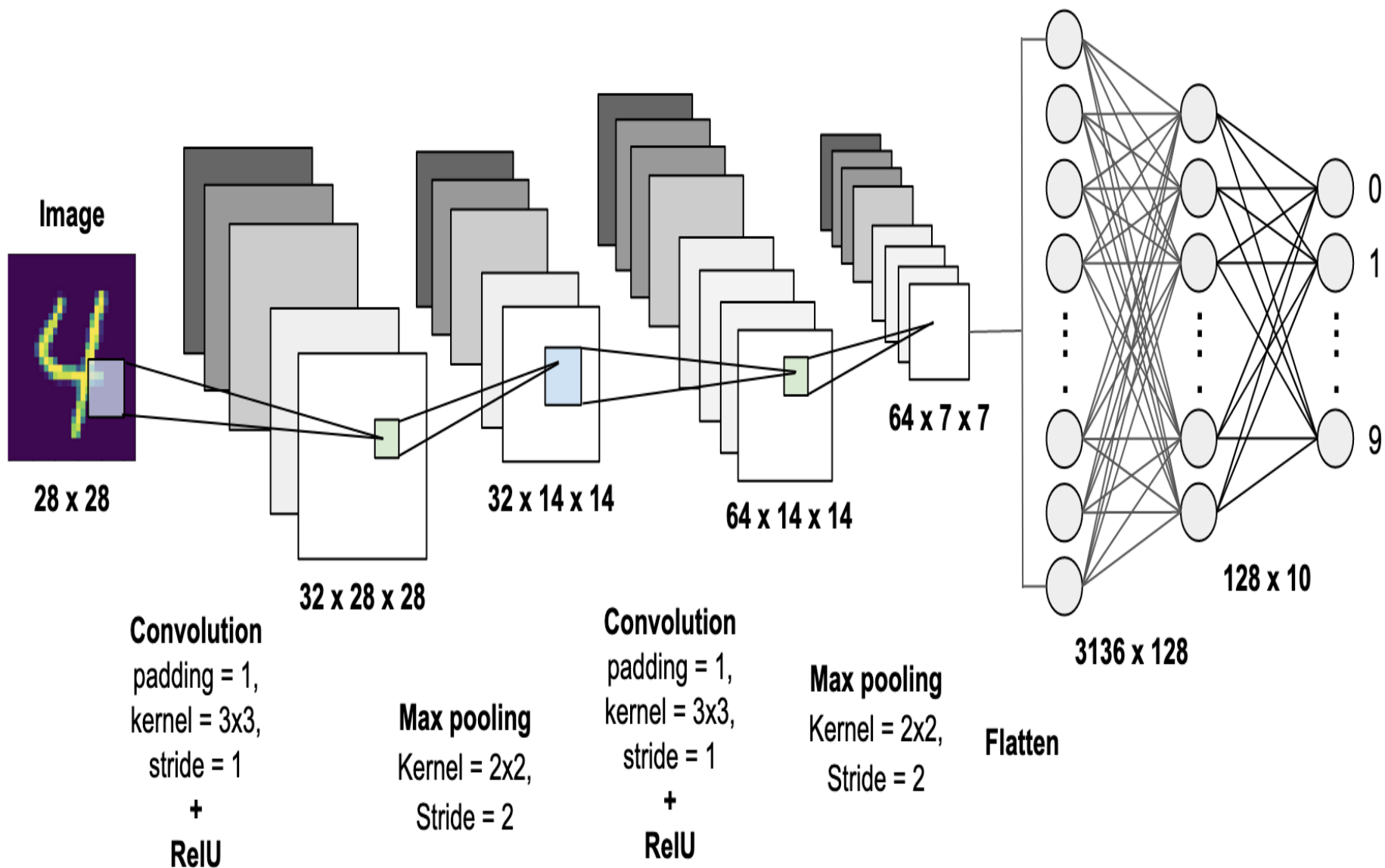
    return output
```

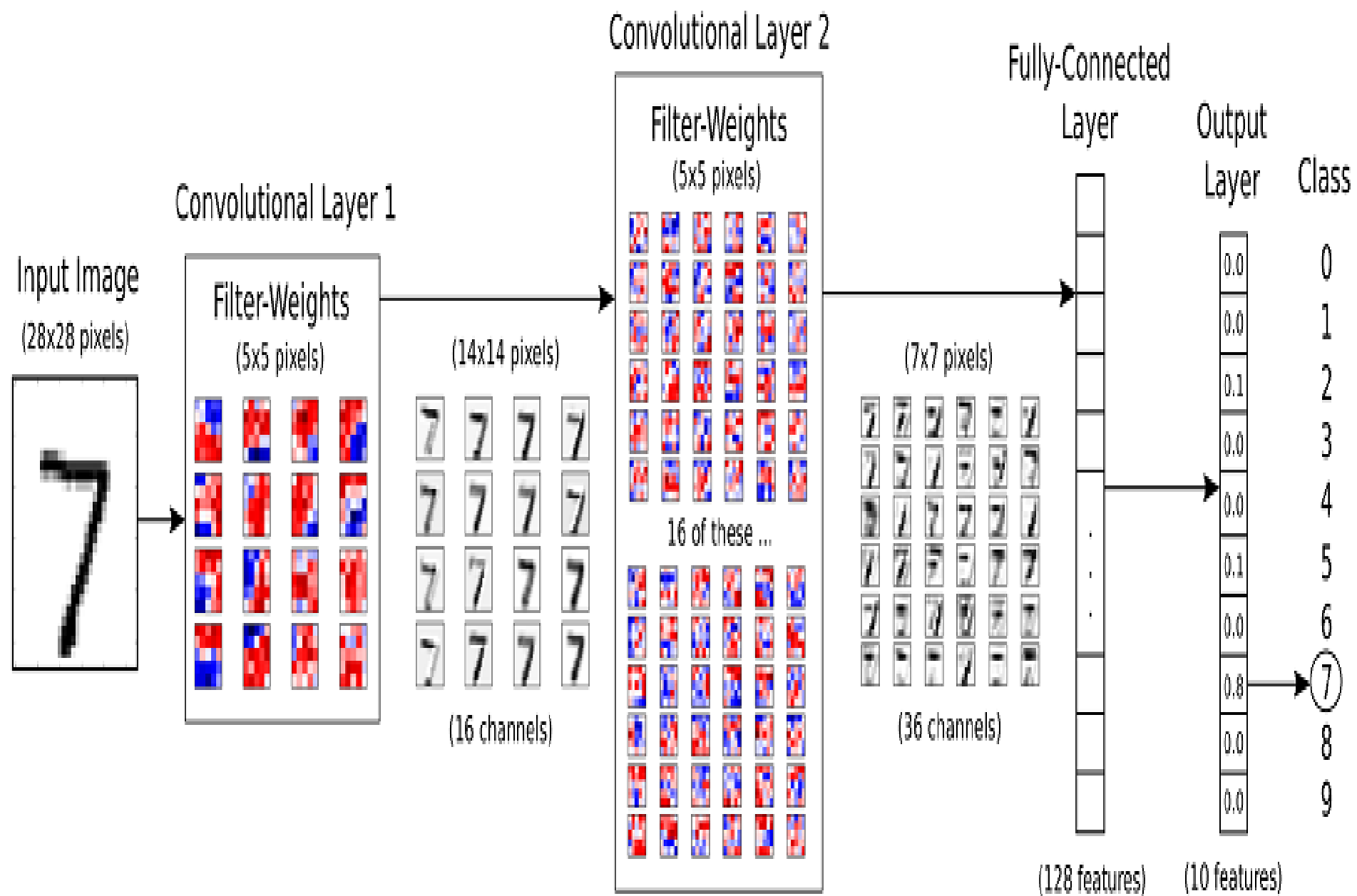
The code here is quite easy to follow. We first take the flattened versions of the input

Simple CNN









label = 5



label = 0



label = 4



label = 1



label = 9



label = 2



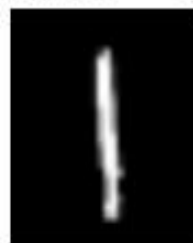
label = 1



label = 3



label = 1



label = 4



label = 3



label = 5



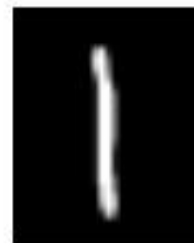
label = 3



label = 6



label = 1



Building a Convolutional Network for CIFAR-10

- The CIFAR-10 challenge consists of 32×32 color images that belong to one of 10 possible classes.

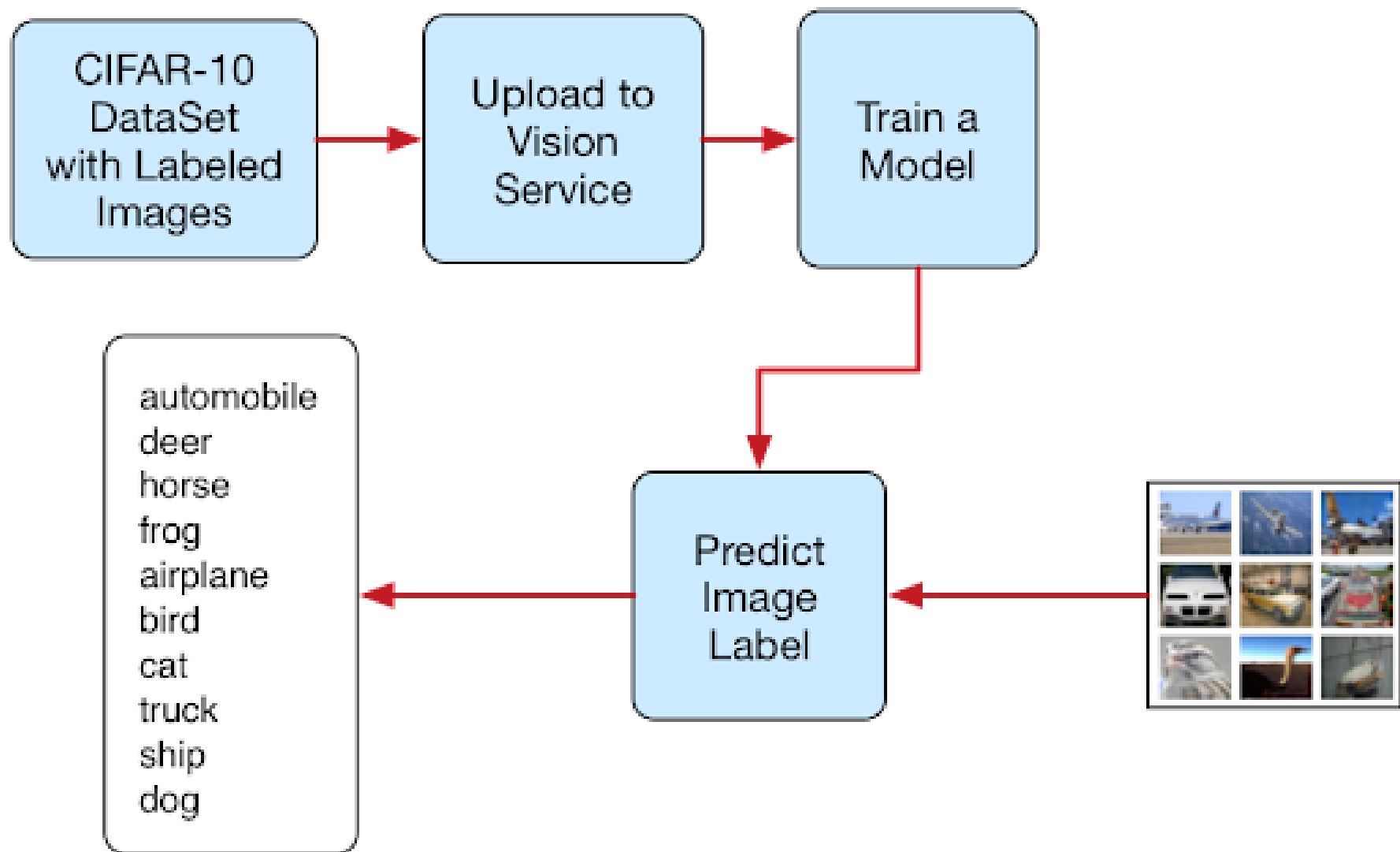




Figure 5-16. A dog from the CIFAR-100 dataset

- build networks both **with and without batch normalization** as a basis of comparison.
- increase the learning rate by **10-fold** for the batch normalization network
- distort random 24×24 crops of the input images to feed into our network for training into the network architecture.
- **integrate** batch normalization into the convolutional and fully connected layers.

```

def conv2d(input, weight_shape, bias_shape, phase_train,
          visualize=False):
    incoming = weight_shape[0] * weight_shape[1]
                    * weight_shape[2]
    weight_init = tf.random_normal_initializer(stddev=
        (2.0/incoming)**0.5)
    W = tf.get_variable("W", weight_shape,
        initializer=weight_init)
    if visualize:
        filter_summary(W, weight_shape)
    bias_init = tf.constant_initializer(value=0)
    b = tf.get_variable("b", bias_shape, initializer=bias_init)
    logits = tf.nn.bias_add(tf.nn.conv2d(input, W,
        strides=[1, 1, 1, 1], padding='SAME'), b)
    return tf.nn.relu(conv_batch_norm(logits, weight_shape[3],
        phase_train))

def layer(input, weight_shape, bias_shape, phase_train):
    weight_init = tf.random_normal_initializer(stddev=
        (2.0/weight_shape[0])**0.5)
    bias_init = tf.constant_initializer(value=0)
    W = tf.get_variable("W", weight_shape,
        initializer=weight_init)
    b = tf.get_variable("b", bias_shape,
        initializer=bias_init)
    logits = tf.matmul(input, W) + b
    return tf.nn.relu(layer_batch_norm(logits, weight_shape[1],
        phase_train))

```



- There are then two fully connected layers followed by a softmax.
- Dropout is included for reference, but in the batch normalization version, keep_prob=1 during training:

```
def inference(x, keep_prob, phase_train):  
  
    with tf.variable_scope("conv_1"):  
        conv_1 = conv2d(x, [5, 5, 3, 64], [64], phase_train,  
                        visualize=True)  
        pool_1 = max_pool(conv_1)  
  
    with tf.variable_scope("conv_2"):  
        conv_2 = conv2d(pool_1, [5, 5, 64, 64], [64],  
                        phase_train)  
        pool_2 = max_pool(conv_2)  
  
    with tf.variable_scope("fc_1"):
```

```
dim = 1
for d in pool_2.get_shape()[1:].as_list():
    dim *= d

pool_2_flat = tf.reshape(pool_2, [-1, dim])
fc_1 = layer(pool_2_flat, [dim, 384], [384],
             phase_train)

# apply dropout
fc_1_drop = tf.nn.dropout(fc_1, keep_prob)

with tf.variable_scope("fc_2"):

    fc_2 = layer(fc_1_drop, [384, 192], [192], phase_train)

# apply dropout
fc_2_drop = tf.nn.dropout(fc_2, keep_prob)

with tf.variable_scope("output"):
    output = layer(fc_2_drop, [192, 10], [10], phase_train)

return output
```

After some amount of time training, our networks are able to achieve an impressive 92.3% accuracy on the CIFAR-10 task without batch normalization and 96.7% accuracy with batch normalization.

Visualizing Learning in Convolutional Networks

- plot the cost function and validation errors over time as training progresses.

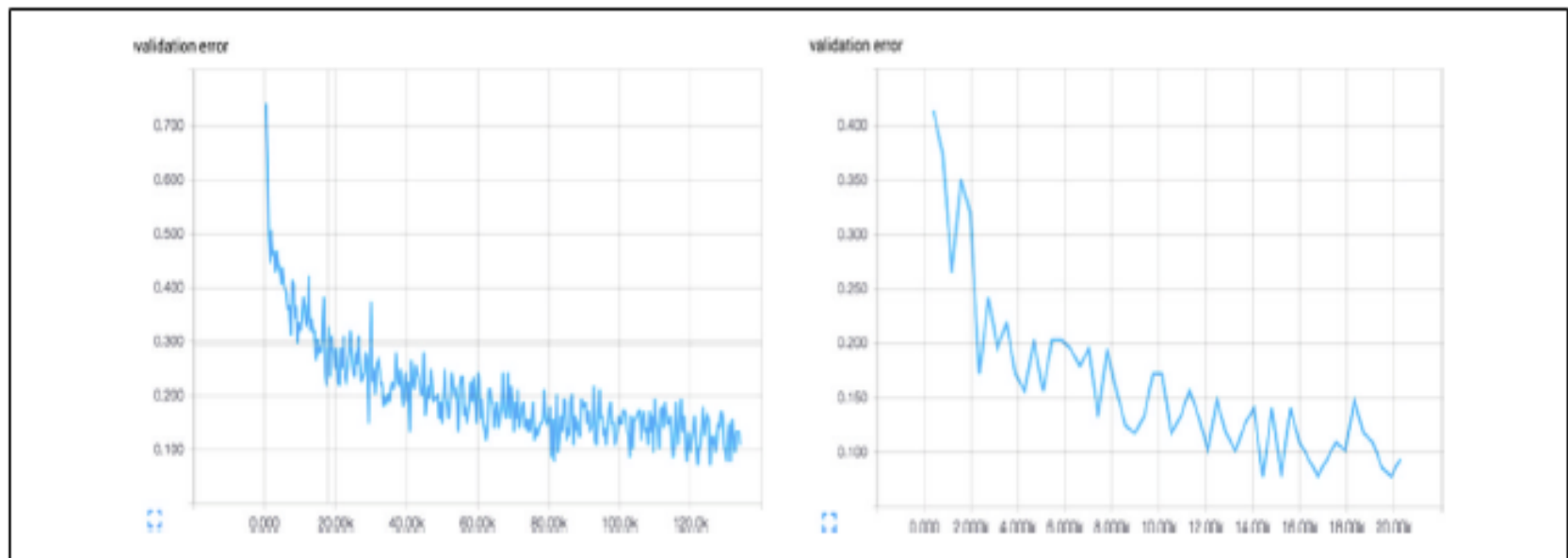


Figure 5-17. Training a convolutional network without batch normalization (left) versus with batch normalization (right). Batch normalization vastly accelerates the training process.

- batch normalization requires 14,000 mini-batches.
- We can also inspect the filters that our convolutional network learns in order to understand what the network finds important to its classification decisions.
- Convolutional layers learn **hierarchical representations**, and so the first convolutional layer learns basic features (**edges, simple curves, etc.**), and the second convolutional layer will learn more complex features.
- Second convolutional layer is difficult to interpret even if we decided to visualize it, so we only include the first layer filters.

- We can make out a number of interesting features in our filters: vertical, horizontal, and diagonal edges, in addition to small dots or splotches of one color surrounded by another.
- We can be confident that our network is learning relevant features because the filters are not just noise.

Leveraging Convolutional Filters to Replicate Artistic Styles

- The goal of neural style is to be able to take an arbitrary photograph and rerender it as if it were painted in .
- clever manipulation of convolutional filters can produce spectacular results on this problem.

Pre-trained convolutional network.

- There are three images that we're dealing with.
- The first two are the source of content ***p and the source of style a.***
- ***The third*** image is the generated image ***x.***
- ***Our goal will be to derive an error function that we*** can backpropagate that, when minimized, will perfectly combine the content of the desired photograph and the style of the desired artwork.

We start with content first. If a layer in the network has k_l filters, then it produces a total of k_l feature maps. Let's call the size of each feature map m_l , the height times the width of the feature map. This means that the activations in all the feature maps of this layer can be stored in a matrix $F^{(l)}$ of size $k_l \times m_l$. We can also represent all the activations of the photograph in a matrix $P^{(l)}$ and all the activations of the generated image in the matrix $X^{(l)}$. We use the relu4_2 of the original VGGNet:

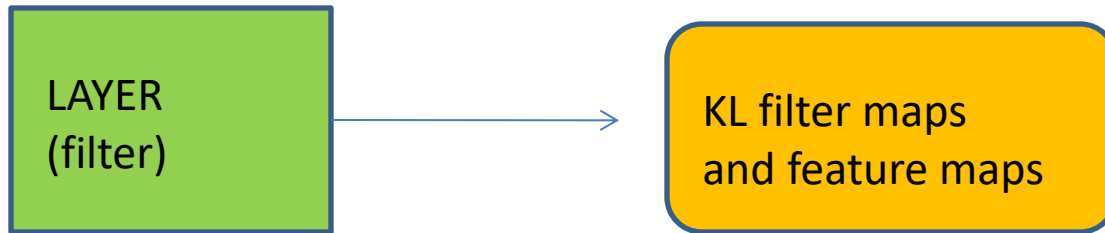
$$E_{\text{content}}(p, x) = \sum_{ij} (P_{ij}^{(l)} - X_{ij}^{(l)})^2$$

Now we can try tackling style. To do this we construct a matrix known as the *Gram matrix*, which represents correlations between feature maps in a given layer. The correlations represent the texture and feel that is common among all features, irrespective of which features we're looking at. Constructing the Gram matrix, which is of size $k_l \times k_l$ for a given image is done as follows:

$$G_{ij}^{(l)} = \sum_{c=0}^{m_l-1} F_{ic}^{(l)} F_{jc}^{(l)}$$

We can compute the Gram matrices for both the artwork in matrix $A^{(l)}$ and the generated image in $G^{(l)}$. We can then represent the error function as:

$$E_{\text{style}}(\mathbf{a}, \mathbf{x}) = \frac{1}{4k_l^2 m_l^2} \sum_{l=1}^L \sum_{ij} \frac{1}{L} (A_{ij}^{(l)} - G_{ij}^{(l)})^2$$



Size of each feature map = m_l

activations of the photograph in a matrix $P^{(l)}$

activations of the generated image in the matrix $X^{(l)}$

$$E_{\text{content}}(p, x) = \sum_{ij} (P_{ij}^{(l)} - X_{ij}^{(l)})^2$$

construct a matrix known as the *Gram matrix*, $k_l \times k_l$,

represents correlations between feature maps in a given layer.

$$\mathbf{G}^{(l)}_{ij} = \sum_{c=0}^{m_l-1} \mathbf{F}^{(l)}_{ic} \mathbf{F}^{(l)}_{jc}$$

ERROR RATE

$$E_{\text{style}}(\mathbf{a}, \mathbf{x}) = \frac{1}{4k_l^2 m_l^2} \sum_{l=1}^L \sum_{ij} \frac{1}{L} (A_{ij}^{(l)} - G_{ij}^{(l)})^2$$

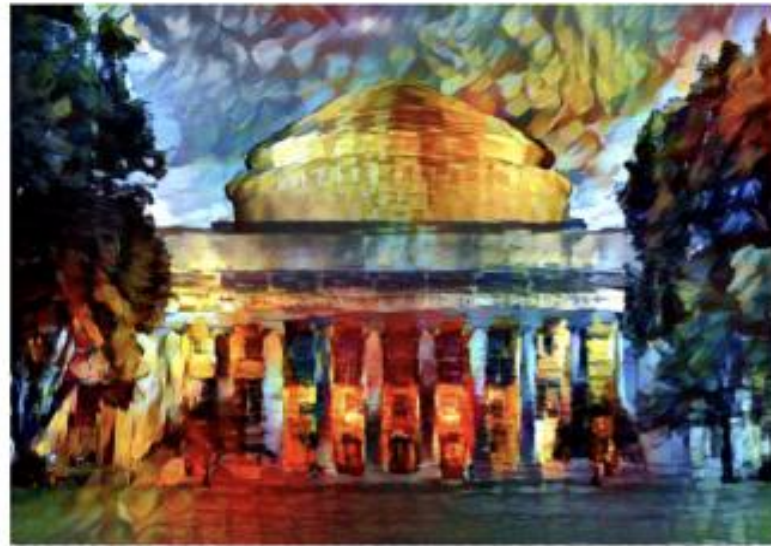


Figure 5-20. The result of mixing the Rain Princess with a photograph of the MIT Dome.
Image credit: Anish Athalve.